Lab 4: Traffic Simulation

Fundamentals of Programming 1.5

Computer system simulations are an important tool for working out a variety of engineering and scientific tasks. In this lab, you will build a simple traffic simulation to identify potential improvements for the flow of cars through a series of intersections. You will work with objects and classes to model traffic lights and cars.

Table of Contents

- Lab Guidelines
- Lab Setup
- Grading
- Your Task
  - Part 1: Light
  - Part 2: Car
  - Part 3: Simulation
- Acknowledgments

Lab guidelines

Academic integrity

Your programs must be your work (or your pair's work, if working in a pair). You may use all permitted *learning* resources, but the code you submit should be written by you. Be prepared to explain the logic and functionality of your code.

Important: Using help sites such as StackOverflow or generative AI tools is not permitted. These resources can be helpful "once you know what you're doing," but they create a difficult grey area while learning. Instead, use Ed discussion, office hours, and talk (without sharing exact code) with other students.

Submitting to Gradescope

Submit the required Python files with the exact filenames listed in this lab. Each submitted `.py` file must include a header docstring with the author(s), the date you started, and a 1–2 sentence summary of what the file does:

```
"""
Author(s): YOUR NAME(S) HERE
Date started: YYYY-MM-DD

1-2 sentence description of this file.
"""
```

Gradescope will run automated tests on your submission and report which tests passed. If you do not receive full credit, you may revise and resubmit before the deadline. Some labs require specific function/class names or signatures for the autograder to run; those requirements will be stated in the writeup.

Some tests are hidden and don't provide feedback. This encourages you to test beyond the visible Gradescope tests.

Code Quality

After the late deadline for each lab, a PM or TA will read through your code and apply manual grading points based on your code and testing quality. The specific point allocation and expectations for code quality are described near the beginning of each lab document, but are generally few points and loose requirements. Our goal is to provide basic feedback for notably concerning code quality problems (especially something called code smells).

Generally, for CS 150, you are not required to adhere to a specific style — good grades for code quality will generally be given for code that uses descriptive variable names, comments to explain confusing code, and is not "excessively complex" (a subjective metric we include to be able to take points off for unusually confusing or convoluted code). If you prefer having the rigor and certainty of a proper style guide, we will generally aim to conform to the Google code style guide for this course.

Pair programming

You may work with a partner on all problems, following pair programming rules: both submit the same code (listing your partner's name in the header), and both receive the same grade. Work on one computer in the same physical location—splitting up work is not permitted. Switch `driver` (typing) and `observer` (reviewing) roles for every problem.

When submitting to Gradescope, indicate that it's a group submission.

Late work

Gradescope accepts late submissions for up to **2 days** after the deadline. It automatically deducts **5% per late day** (no need to notify us separately). It considers the lab as a whole (if one required file is late, the entire lab is marked late), but it is still usually worth submitting as much as you can.

Lab Setup

Start by downloading the .zip archive and unzipping it to a folder on your computer (if you've never unzipped a folder before, check out this link for Windows and this link for Mac, or search for an article that works for you).

All student work for this lab happens in the `student/` folder. For your code to run correctly, make sure the provided `.py` files stay together in that folder, and run commands from there.

File Name	Description
lab4.md	this writeup
simulation.py	logic for your simulation
test.py	test cases for the simulation
graph.py	optional script for exploring simulation results (e.g., printing summary data)
example_output1.txt	first example output expected from running your simulation
example_output2.txt	second example output expected from running your simulation
example_output3.txt	third example output expected from running your simulation

You will submit the following files to Gradescope:

File Name	Description
simulation.py	Your finished traffic simulation logic
test.py	Your finished collection of tests for the traffic simulation

Your goals in this lab: 1. Implement tests for new methods in test.py 2. Implement all unimplemented methods in simulation.py

How to run your code

Do all your work from the `student/` folder. Make sure `simulation.py`, `test.py`, and `graph.py` are all in the same folder.

Run the tests:

```
python3 test.py
```

- Run the simulation demo (this writes `simulation_result.txt`):

```
python3 simulation.py
```

How to read the sample output files

The sample output files (`example_output1.txt`, etc.) show `str(simulation)` for the solution at specific timesteps. Read **top to bottom**:

`START` marks the **incoming** side (before Light 1).

`FINISH` marks the end (after the last light).

For each light segment:

- Cars printed **before** the `...` separator are **traveling toward the next light**.
- Cars printed **after** the `...` separator (and before the light line) are **queued at that light**.
- The light prints as `Light (id: N): Green` or `Light (id: N): Red`.

Grading

Grading for this lab is divided as follows:

- 30%: initial tests provided and those visible on Gradescope for simulation.py
- 55%: pass the "hidden" tests
- 10%: at least **1 additional** test case for **each provided test function** (that is, one test case for each TODO comment in test.py). Note that this involves writing fewer test cases than in some previous labs, but these test cases may be more involved.
- 2%: including your name and the date on *each file*
- 3%: reasonable code style, including descriptive variable names and solutions that are not "excessively" complex. Examples of an "excessively complex" solution could include many (10+) nested if statements, or many loops when only one or two are needed.

Test cases are graded only on existence and correctness (your code should pass your own test cases!). We suggest writing comments explaining testing edge cases or clever coding tricks, but do not require it.

Your task

Our goal in this lab is to implement a simple traffic stop simulation to identify parameters we can use to improve the flow of traffic. We are interested in traffic flow through a series of traffic lights and in places where traffic might get especially "stuck" depending on traffic light timing. Since this is a simple simulation of a quite complicated problem, we make several simplifying assumptions:

- We only simulate cars going in a single direction through a fixed series of intersections (cars never turn and must go through all traffic lights in sequence)
- Traffic moves in discrete units of time called a *timestep*. Each timestep is equivalent to about 3 seconds in the real world.
- Traffic lights are red for the same amount of time as they are green and alternate at a fixed rate
- There are no other obstacles; all cars move between traffic lights at a similar rate
- Only one car may pass through a given traffic light at a time

We will measure the average number of cars at each traffic light and adjust the length of time that each light remains red or green to try to improve traffic flow.

Your task is divided into 3 parts: 1. Test and implement the Light class to simulate a simple traffic light 2. Test and implement the Car class to simulate simple car behavior 3. Test and implement the Simulation class to construct and run a simple traffic simulation

NOTE: Unlike in previous labs, the specifications describe behaviors of a class rather than return values of a function. This kind of thinking can take some getting used to but is important when working with objects (and complex data in general).

The upside is that you have some flexibility in how you implement each class — so long as your objects and their methods behave as specified, your code is considered correct. The same applies to the test cases: they focus on testing the behavior of your objects rather than a single function's

return value. So tests are spread across a few scenarios; reading through them can be more involved but also gives a lot of information.

NOTE: If you are writing a lot of code for this simulation, you might be on the wrong track. Our solution for simulation.py adds only about 60 lines of code beyond the starting point (including extra spacing). If you feel you need to add a lot of code, we suggest coming to office hours to discuss your approach.

Part 1: Light

The Light class represents a single traffic light in our simulation. Each Light maintains state for switching between red and green and a queue of cars currently waiting at that Light. "Queue" means the first car to arrive at a traffic light is also the first to pass through it.

We provide a collection of attributes (with type hints) for you to use when implementing the Light class. You may add more attributes if helpful, but you are not required to.

We also provide a constructor and `str` / `repr` implementations for the Light class. These methods are useful when debugging — you can use:

```
light = Light(5, 3)
print(str(light))  # The pretty string representation of this light
print(repr(light)) # The debugging information about this light
```

First, look at the provided test cases for the Light class and add a simple test case for each test function we provide (under each TODO comment for the light). Your additional test for each does not need to be complicated; the goal is to convince yourself you understand what each method in Light is supposed to do.

Once you understand the behavior of a Light, implement all methods marked with `# TODO: Implement me!`

Part 2: Car

The Car class represents a single car passing through our traffic simulation. Implementing the Car class should follow a similar flow to implementing the Light class.

Three details to keep in mind:

- The syntax `current_light: Light | None` means `current_light` can be a Light or `None`. When using properties of `current_light`, first check that it is not `None`.
- The `start_going` method of Car relies on two constants defined at the top of the file. We have seen such constants in earlier labs; here we will validate through testing that you use them directly. Use these constants rather than hard-coding "magic numbers", since unexplained numbers in code are bad style and can lead to bugs.
    - A "magic number" is jargon for any number that appears in code without explanation.

    - For example, `self.travel_time = TRAVEL_TIME` is acceptable.
    - In contrast, `self.travel_time = 15` uses a magic number and is bad style.
- You need to introduce randomness so that cars have some variation when traveling between lights. You should use random.randint, as in the recent Coin simulation activity. Note that `random.randint(0, 5)` returns a random integer from 0 to 5 inclusive.

Part 3: Simulation

With the Car and Light classes implemented, we can implement the simulation. The `timestep` function of the simulation is likely the trickiest part of this lab, because many things happen in one step. The simulation timestep also has fairly basic test cases — with so much going on, very precise test cases tend to cause more confusion than they help.

To help you check that your simulation is correct, we provide several text files showing the output of the solution version of simulation.py in a few situations. Compare your simulation to these files by running simulation.py and inspecting the generated simulation_result.txt file (which is created from the `__str__` method of the Simulation class).

The example files were produced with the following parameters:

File Name	Number of Lights	Light Timestep Change	Light "offset"	Spawning Time	Simulation Timestep
example_output1.txt	3	5	1	3	1
example_output2.txt	3	5	1	3	11
example_output3.txt	4	5	2	5	12

If your simulation does not match ours, double-check your test cases for the Light and Car classes to make sure you have tested them sufficiently. If you are confident those are correct, then look at your implementation of the simulation timestep in more detail. A common mistake is forgetting to use the intended randomness in Car or in the Simulation spawn_rate; make sure you use the correct range for each of these values.

NOTE: The example outputs are also meant to help you visualize what the simulation is doing. The `__str__` method of the Simulation class describes how these files are built; you may find that helpful when forming a mental model of how the timestep should work.

Acknowledgments

The lab is authored by Dietrich Geisler with edits by Anastasia Kurdia.